ClickLess AI: A Conversational Agent for Autonomous Online Grocery Shopping

Swayam Atul Mehta
Arizona State University
smehta90@asu.edu

Chirag Manjeshwar
Arizona State University
cmanjesh@asu.edu

Aviral Gupta
Arizona State University
agupt467@asu.edu

Amrit Vinod Mahajan
Arizona State University
amahaj56@asu.edu

Shashwat Singh Chauhan
Arizona State University
schauh47@asu.edu

Naman Dewan
Arizona State University
ndewan1@asu.edu

***Abstract*—ClickLess AI is a conversational agent system that automates online grocery shopping through natural language interaction. A user describes their grocery needs in plain language; the system queries the Instacart Developer Platform API, structures retrieved product data into a knowledge graph enriched with Open Food Facts nutritional attributes, ranks results using a combination of classical data mining baselines and LLM-powered scoring and returns a checkout-ready cart requiring minimal user effort. All credential operations and simple queries are handled by a locally-hosted Llama 3.2 11B instance, while complex queries and vision tasks are delegated to a quantized Llama 3.2 90B model via Ollama. This ensures sensitive data never leaves the device. Browser automation via BrowserUse and Playwright is reserved exclusively for the final checkout handoff, after all product decisions have been made through the API layer. The system is grounded in verified open APIs, published datasets and concrete evaluation targets, designed to be fully implemented within the semester timeline.**

***Index Terms*—Conversational AI, Knowledge Graph, LLM Orchestration, Instacart API, BrowserUse, Playwright, Llama 3, Multi-Agent Systems, Grocery Automation, ReAct, GraphRAG, Association Rule Mining**

I. Problem Definition

Online grocery shopping, despite widespread adoption, remains a manual and repetitive task. Consumers must navigate platform-specific interfaces, compare prices across separate sessions, remember preferences between visits and re-authenticate on every device. A 2025 CapitalOne study found U.S. consumers spend over 65 hours annually on grocery shopping [1]. ClickLess AI addresses this gap by converting a single natural-language utterance into a populated, checkout-ready grocery cart.

Three specific technical challenges shape the system architecture:

1) **Credential security.** Browser automation pipelines that pass login credentials through cloud-hosted LLMs expose sensitive user data to third-party servers. We isolate all credential operations to on-device Llama 3.2 models, so passwords and session tokens never leave the local machine.
2) **Platform reliability.** High-value grocery retailers such as Amazon Fresh and Walmart Grocery deploy Cloudflare and Akamai bot-detection infrastructure. Browser scraping against these platforms is both technically unreliable and legally inappropriate for academic use. Our system builds exclusively on the official Instacart Developer Platform API for all product discovery and cart construction.
3) **Preference learning without remote storage.** The agent must accumulate individual preferences (preferred brands, dietary restrictions, price thresholds) across sessions without transmitting a user profile to external services. A locally persisted preference model, updated from the Instacart 2017 order history and in-session user selections, resolves this.

II. Datasets

A. Primary Live Data: Instacart Developer Platform API

The Instacart Developer Platform API [2] provides real-time access to product search, store availability, pricing, retailer locator and cart creation across Instacart-connected retailers. It explicitly supports LLM and AI agent integration and requires no scraping or reverse engineering. This serves as the system's primary data backbone.

B. Behavioral Dataset: Instacart 2017

The Instacart Online Grocery Shopping Dataset [3] contains over 3 million anonymized orders from more than 200,000 users, including product IDs, order sequences, reorder indicators, day-of-week and hour-of-day features. Product IDs from this dataset are not joined to the live API; only aggregate behavioural patterns are used: co-purchase association rules, reorder rates and order timing signals. This dataset seeds the knowledge graph with real purchase co-occurrence patterns, provides training data for the Apriori and logistic regression baselines and initializes the on-device user preference module.

C. Semantic Dataset: Open Food Facts

Open Food Facts [4] is a free, crowdsourced database of over 3 million food products containing nutritional data, allergen information, ingredient lists, NOVA processing categories and Nutri-Score ratings. It enriches the knowledge graph with semantic product attributes, enabling the agent to

satisfy dietary restrictions (e.g., "gluten-free", "low sodium") and nutritional goals directly from the graph without additional LLM calls.

D. Web Automation Benchmark: MiniWoB++

MiniWoB++ [5] is a collection of over 100 simulated web interaction tasks including button clicks, text entry, form fills and multi-step navigation, used to benchmark the action reliability of browser agents. We use a subset of form-fill and navigation tasks to validate the BrowserUse checkout handoff component prior to deployment on live Instacart pages.

III. State-of-Art Methods & Algorithms

A. System Architecture Overview

Fig. 1 illustrates the end-to-end pipeline. A natural language query enters the NLU layer, is decomposed into structured slots and routed through a LangGraph multi-agent orchestration loop. The Instacart API agent fetches live product data; the Knowledge Graph layer enriches and ranks candidates; the local Llama 3 instance manages credentials; and BrowserUse executes the checkout handoff.

B. Knowledge Graph Construction

The knowledge graph is built using LLM-powered Subject-Predicate-Object (SPO) triple extraction following the pipeline described in [6]. Product descriptions and Open Food Facts records are chunked into 200-300 word segments with 10% overlap to preserve boundary context. Mistral 7B processes each chunk, extracting triples with predicates constrained to 1-3 words (e.g., *oat milk* $\rightarrow$ *contains* $\rightarrow$ *beta-glucan*). An entity standardisation step uses LLM-assisted clustering to merge name variants ("oat milk", "oatmilk", "oat-milk" $\rightarrow$ canonical form). Transitive and lexical-similarity inference then bridges disconnected subgraphs. All triples are persisted in Neo4j and queried via Microsoft GraphRAG [7], which returns explainable, source-cited answers grounded in actual product records rather than LLM parametric memory.

C. Classical Baselines (DM/ML)

Two classical baselines are implemented for comparison per the course requirement to include both classical and modern methods:

- **Apriori association rule mining** on the Instacart 2017 order history discovers frequently co-purchased product pairs (support ≥ 0.01, confidence ≥ 0.3). These rules power the "frequently bought together" recommendations and serve as Config A in the ablation study.
- **Logistic regression ranker** scores candidate products on four normalised features: price relative to user budget, in-stock probability, query-term match (TF-IDF cosine) and historical reorder rate from the Instacart 2017 dataset. This forms the ranking baseline before KG enrichment.

D. Agentic Orchestration: ReAct

The ReAct framework [8] (ICLR 2023) interleaves reasoning traces and external tool calls in a loop, enabling the agent to dynamically replan when conditions change. For example, when a preferred item is out of stock, the agent reasons about substitutes, calls the Instacart API for alternatives and updates its plan without user intervention. LangGraph manages the directed agent graph: NLU Agent $\rightarrow$ Instacart API Agent $\rightarrow$ KG Ranking Agent $\rightarrow$ Browser Handoff Agent.

E. Browser Automation

BrowserUse [9] drives a hybrid DOM-plus-screenshot interaction model that does not rely on hard-coded CSS selectors, making it robust to minor page layout changes. It controls Playwright [10] for the checkout step. Browser automation is strictly limited to the final handoff after the API layer has constructed the cart, minimising exposure to bot-detection systems. Session tokens generated on-device by Llama 3.2 11B are injected directly, eliminating the need to replay the login flow in the browser.

F. Local LLM for Privacy

Llama 3.2 11B and a quantized Llama 3.2 90B [11] run entirely on-device via Ollama. Llama 3.2 11B handles simple queries, manages the encrypted credential vault, generates session tokens for the browser handoff, and persists the per-user preference profile. The quantized Llama 3.2 90B handles complex queries and provides vision capabilities crucial for interacting with BrowserUse. Because these models never contact an external endpoint, credentials and purchase history remain private by construction. Mistral 7B [12] (Midweight Tier; $\sim$6 GB RAM) handles NLU intent extraction and SPO triple extraction, chosen for its strong instruction-following performance at this parameter count.

IV. Research Plan

The project is structured in three phases that incorporate both classical DM/ML baselines and modern AI methods, aligned with the course milestone calendar.

A. Phase 1: Infrastructure & Classical Baselines (Feb 25 - Mar 22)

- Register and validate the Instacart Developer Platform API. Document all endpoint capabilities, rate limits and authentication flows.
- Collect and preprocess the Instacart 2017 dataset and Open Food Facts data. Define a unified canonical product schema mapping both sources to common fields: `name`, `price`, `availability`, `platform`, `fetch_timestamp`, `nutriscore`, `allergens`.
- Implement Apriori association rule mining on Instacart 2017 order history. Tune support and confidence thresholds. Implement logistic regression ranker on the four-feature product vector. Establish MiniWoB++ action reliability baseline without KG.

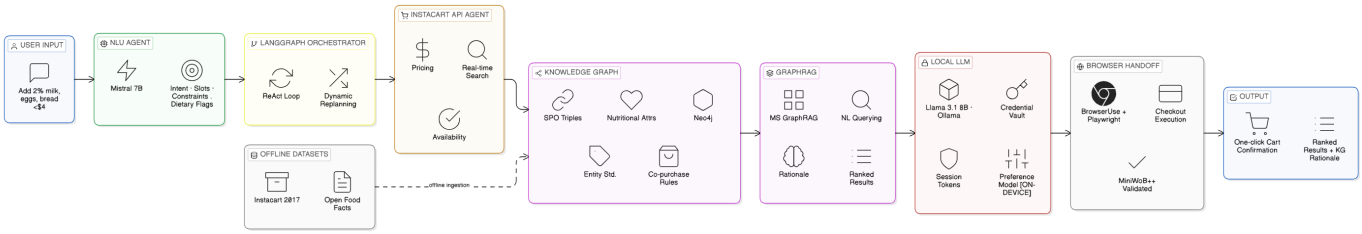

Fig. 1. ClickLess AI system architecture. Solid arrows show the live query flow; dashed arrow indicates offline dataset ingestion into the Knowledge Graph.

- Install Llama 3.2 11B, quantized Llama 3.2 90B, and Mistral 7B locally via Ollama. Implement the encrypted credential vault and session token generation module. Verify end-to-end local inference latency.
- Build a Streamlit UI stub: conversational text input, result list with price and availability, cart confirmation button.

B. Phase 2: KG Construction & Agent Integration (Mar 23 - Apr 12)

This phase is timed to the class presentations, during which a working API-plus-KG demo will be shown.

- Run SPO triple extraction (Mistral 7B) over all product descriptions and Open Food Facts records. Load into Neo4j with entity standardisation and transitive relationship inference. Verify graph connectivity and triple precision via manual spot-check.
- Integrate Microsoft GraphRAG for natural language KG querying. Test explainability of rationale snippets on a set of 20 standardised queries.
- Deploy the full multi-agent pipeline via LangGraph: NLU Agent parses the user query; Instacart API Agent fetches candidates; KG Ranking Agent traverses Neo4j and scores results; Browser Handoff Agent executes checkout using Llama 3-issued session tokens.
- Implement the preference learning module: update the local user profile after each confirmed cart based on selected items, rejected suggestions and filter adjustments.
- Extend Streamlit UI with a PyVis knowledge graph visualisation panel showing the product entity graph for the current query.

C. Phase 3: Evaluation & Demos (Apr 13 - May 4)

This phase spans the group demo window and concludes with the final report.

- Run the full ablation study across Configs A, B and C (Section V). Report Task Success Rate, Constraint Satisfaction Score, NDCG@5 and Clicks Saved for each configuration.
- Evaluate BrowserUse checkout handoff on the MiniWoB++ form-fill and navigation tasks. Report Execution Success Rate.
- Conduct a mock user study with 5-7 participants on three standardised shopping scenarios (weekly grocery list, dietary-restricted meal prep, budget-capped bulk purchase). Collect task completion rate, clicks saved and System Usability Scale (SUS) score.
- Red-team edge case testing: out-of-stock substitution, price change between query and checkout, CAPTCHA encounter in browser handoff, malformed API response handling.
- Write final paper, record demo video and release code with documentation.

V. Evaluation Plan

A. Task Quality Metrics

Task Success Rate (TSR) measures the percentage of natural-language shopping requests for which the agent returns at least one valid, in-stock result satisfying all hard constraints (item type, price ceiling, dietary flags, quantity). A task is considered successful only if all hard constraints are met. Target: $\geq 85\%$.

Constraint Satisfaction Score (CSS) measures the fraction of explicitly stated user constraints present in the top-ranked returned result, averaged over all test queries. Target: $\geq 90\%$.

NDCG@5 measures the normalised discounted cumulative gain of the top-5 returned results against a human-annotated relevance ranking, evaluating the ordering quality of the KG ranker versus the logistic regression baseline.

B. Efficiency Metrics

Clicks Saved counts the difference between the number of UI interactions required for a manual baseline shopping session and the number required using ClickLess AI for an equivalent 10-item grocery list. Target: ≥ 10 clicks saved per session.

Time-to-First-Option measures wall-clock time from query submission to display of the first ranked result. This captures end-to-end API latency plus NLU and KG traversal time. Target: < 5 seconds.

C. Knowledge Graph Quality

Triple Precision is measured by manually verifying a random sample of 100 extracted SPO triples against their source product records. A triple is marked correct if both entities and the predicate accurately reflect the source text.

Explanation Quality rates the clarity and factual accuracy of GraphRAG-generated recommendation rationale on a 5-point Likert scale collected in the user study (1 = confusing / incorrect, 5 = clear / accurate).

D. Browser Agent Reliability

Execution Success Rate (ESR) is evaluated on the MiniWoB++ benchmark subset of form-fill and navigation tasks. It is defined as the ratio of successfully completed action sequences to optimal action sequences for the same tasks. Target: $\geq 90\%$.

E. Ablation Study

We will be performing ablation studies under three different configurations:

1) **Classical baselines only (Apriori + logistic regression)**: Lower-bound performance; no LLM reasoning or KG
2) **ReAct agent + Instacart API; no KG**: Isolates the contribution of KG-RAG enrichment over raw API results
3) **Full system: ReAct + KG-RAG + GraphRAG rationale**: Proposed system; expected highest TSR, CSS and NDCG@5

VI. Project Timeline

Period	Tasks	Milestone
Feb 25	API registration; local LLM setup (Llama 3.2 models, Mistral 7B); dataset collection; Apriori + logistic regression baselines; product schema; Streamlit stub	**Proposal due**
Feb 26 - Mar 22	SPO triple extraction pipeline; Neo4j KG construction; entity standardisation; relationship inference; LangGraph orchestration; Instacart API agent	KG + API pipeline ready
Mar 23 - Apr 8	GraphRAG integration; BrowserUse + Playwright checkout; session token injection; preference learning; Streamlit KG visualisation; end-to-end system demo	**Class presentations**
Apr 9 - 12	Ablation study (A/B/C); MiniWoB++ browser evaluation; robustness testing; UI polish and performance tuning	Pre-demo freeze
Apr 13 - 29	Mock user study (5-7 participants); red-team edge cases (out-of-stock, stale price, CAPTCHA); metric collection and analysis	**Group demos**
Apr 30 - May 4	Final paper; demo video recording; code documentation and public release	**Final report**

VII. Division of Work

Each team member owns one core technical component end-to-end. Evaluation framework, ablation study and user study are shared equally across all members.

Member	Component	Responsibilities
Shashwat Singh Chauhan	Knowledge Graph	SPO triple extraction pipeline (Mistral 7B); Neo4j setup and schema; entity standardisation; transitive relationship inference; PyVis graph visualisation
Naman Dewan	API & Orchestration	Instacart Developer Platform API integration; canonical product schema; LangGraph multi-agent pipeline
Aviral Gupta	NLU & Intent	Mistral 7B NLU setup via Ollama; intent classification; slot filling (item, quantity, price, dietary flags); dialogue state management; query constraint parser
Amrit Vinod Mahajan	Classical Baselines & Data	Apriori association rule mining on Instacart 2017; logistic regression ranker; Open Food Facts preprocessing; feature engineering; baseline metric benchmarking
Chirag Manjeshwar	Local LLM & GraphRAG	Llama 3.2 models setup via Ollama; encrypted credential vault; session token generation; user preference learning module; GraphRAG integration and rationale quality evaluation
Swayam Atul Mehta	Browser Handoff	BrowserUse + Playwright checkout execution; Llama 3 session token injection; MiniWoB++ action reliability evaluation; anti-bot robustness testing; Streamlit UI integration

References

[1] Capital One Shopping, "Grocery Shopping Statistics: Behavior & Demographics," 2025. [Online]. Available: https://capitaloneshopping.com/research/grocery-shopping-statistics/

[2] Instacart, "Instacart Developer Platform API," 2024. [Online]. Available: https://docs.instacart.com/developer_platform_api/

[3] Instacart, "The Instacart Online Grocery Shopping Dataset 2017," Kaggle, 2017. [Online]. Available: https://www.kaggle.com/competitions/instacart-market-basket-analysis/data

[4] Open Food Facts, "Open Food Facts Database," 2024. [Online]. Available: https://world.openfoodfacts.org/data

[5] E. Z. Liu, K. Guu, P. Pasupat, T. Shi and P. Liang, "Reinforcement Learning on Web Interfaces using Workflow-Guided Exploration," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2018. [Online]. Available: https://arxiv.org/abs/1802.08802

[6] R. McDermott, "AI-Knowledge-Graph: LLM-powered knowledge graph pipeline," GitHub, 2024. [Online]. Available: https://github.com/robert-mcdermott/ai-knowledge-graph

[7] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt and J. Larson, "From Local to Global: A Graph RAG Approach to Query-Focused Summarization," *arXiv:2404.16130*, 2024. [Online]. Available: https://arxiv.org/abs/2404.16130

[8] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan and Y. Cao, "ReAct: Synergizing Reasoning and Acting in Language Models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023. [Online]. Available: https://arxiv.org/abs/2210.03629

[9] Browser Use, "Browser Use: LLM-powered web automation," 2024. [Online]. Available: https://browser-use.com

[10] Microsoft, "Playwright: Fast and Reliable End-to-End Testing for Modern Web Apps," 2024. [Online]. Available: https://playwright.dev/

[11] Meta AI, "Llama 3.2: Connect 2024 – Vision, Edge, and Mobile Devices," 2024. [Online]. Available: https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices/

[12] Mistral AI, "Mistral 7B," 2023. [Online]. Available: https://mistral.ai/news/announcing-mistral-7b/